

Эффективная реализация лямбда-функций в объектно- ориентированных языках программирования

Болохононов Артем

Научный руководитель: Трепаков Иван Сергеевич

Анонимные функции

Определение

Анонимные функции

Определение

- *Анонимные функции* (или *лямбда-функции*) — функции, которые создаются по ходу программы и не имеют уникального идентификатора,

```
val even = { x: Int ⇒ x % 2 == 0 }
```

Анонимные функции

Определение

- *Анонимные функции* (или *лямбда-функции*) — функции, которые создаются по ходу программы и не имеют уникального идентификатора,
- Анонимные функции могут замыкать окружающий контекст, в таком случае они называются *функциональными замыканиями*,

```
val even = { x: Int ⇒ x % 2 == 0 }
```

```
val m: Int = 5  
val leq = { x ⇒ x < m }
```

...

```
leq(x) // == { x < 5 }
```

Анонимные функции

Определение

- *Анонимные функции (или лямбда-функции)* — функции, которые создаются по ходу программы и не имеют уникального идентификатора,
- Анонимные функции могут замыкать окружающий контекст, в таком случае они называются *функциональными замыканиями*,
- Сейчас в объектно-ориентированных языках программирования они используются повсеместно, часто используются при работе с коллекциями.

```
val even = { x: Int ⇒ x % 2 == 0 }
```

```
val m: Int = 5  
val leq = { x ⇒ x < m }
```

...

```
leq(x) // = { x < 5 }
```

```
val collection: Seq[Int] = ...  
collection.filter (x ⇒ x % 2)  
collection.sort (x, y ⇒ x > y)  
collection.foreach(x ⇒ println(x))  
collection.map (x ⇒ x * x)  
...
```

Анонимные функции

В объектно-ориентированных языках программирования

В объектно-ориентированных языках лямбда-функции — это абстракция над синтетическими классами.

```
val capture = "arg = "  
  
val lambda = { x => println(capture + x) }  
lambda(42)
```

Анонимные функции

В объектно-ориентированных языках программирования

В объектно-ориентированных языках лямбда-функции — это абстракция над синтетическими классами.

```
class Lambda$$1(val prefix: Str) extends (Int ⇒ Unit) {  
  override def apply(x: Int) = {  
    println(prefix + x)  
  }  
}  
  
val capture = "arg = "  
val lambda = new Lambda$$1(capture)  
lambda(42)
```

Анонимные функции

В объектно-ориентированных языках программирования

В объектно-ориентированных языках лямбда-функции — это абстракция над синтетическими классами.

```
class Lambda$$1(val prefix: Str) extends (Int ⇒ Unit) {  
  override def apply(x: Int) = {  
    println(prefix + x)  
  }  
}  
  
val capture = "arg = "  
val lambda = new Lambda$$1(capture)  
lambda(42)
```

Таким образом, анонимные функции в объектно-ориентированных языках — это полноценный объект, который требует как аллокации, так и сборки мусора. Из-за чего возникают издержки по памяти и времени.

Постановка задачи

Цель: оптимизация издержек представления анонимных функций в объектно-ориентированных языках программирования.

Постановка задачи

Цель: оптимизация издержек представления анонимных функций в объектно-ориентированных языках программирования.

Задачи:

1. Изучить существующие подходы к уменьшению издержек на выделение объектов в куче,
2. Разработать подход к оптимизации издержек анонимных функций и функциональных замыканий,
3. Выполнить экспериментальную реализацию предложенного подхода,
4. Провести апробацию на представительном наборе тестов и приложений.

Оптимизации

Анализ утеканий

Оптимизации

Анализ утеканий

1. Скаляризация объектов

```
class SomeObject(var a: Int, var b: Double, c: String)

val obj = SomeObject(42, 42.0, "42")
useField(obj.a)
useField(obj.b)
obj.c = "override"
```

Оптимизации

Анализ утеканий

1. Скаляризация объектов

```
class SomeObject(var a: Int, var b: Double, c: String)

val (objA, objB, objC) = (42, 42.0, "42")
useField(objA)
useField(objB)
objC = "override"
```

Оптимизации

Анализ утеканий

1. Скаляризация объектов

```
class SomeObject(var a: Int, var b: Double, c: String)

val (objA, objB, objC) = (42, 42.0, "42")
useField(objA)
useField(objB)
objC = "override"
```

- Данное преобразование может быть применено только на объектах, у которых нет использований по ссылке.

Оптимизации

Анализ утеканий

2. Создание объектов на стеке

```
class SomeObject(var a: Int, var b: Double, c: String)

val obj = SomeObject(42, 42.0, "42")
useField(obj.a)
useField(obj.b)
obj.c = "override"

useObject(obj)
```

Оптимизации

Анализ утеканий

2. Создание объектов на стеке

```
class SomeObject(var a: Int, var b: Double, c: String)

val obj = onStack(SomeObject(42, 42.0, "42"))
useField(obj.a)
useField(obj.b)
obj.c = "override"

useObject(obj)
```


Оптимизации

Анализ утеканий

2. Создание объектов на стеке

```
class SomeObject(var a: Int, var b: Double, c: String)

val obj = onStack(SomeObject(42, 42.0, "42"))
useField(obj.a)
useField(obj.b)
obj.c = "override"

useObject(obj)
```

- Можно применить, если доказанно, что объект существует только в пределах своего стекового кадра.

Оптимизации

Анализ эвакуаций

```
class SomeObject(var a: Int, var b: Double, c: String)

val obj = onStack(SomeObject(42, 42.0, "42"))
useField(obj.a)
useField(obj.b)
obj.c = "override"

otherObject.field = obj // otherObject также находится на стеке
useObject(obj)

...

staticField = heapification(otherObject)
```

Оптимизации

Анализ эвакуаций

```
class SomeObject(var a: Int, var b: Double, c: String)

val obj = onStack(SomeObject(42, 42.0, "42"))
useField(obj.a)
useField(obj.b)
obj.c = "override"

otherObject.field = obj // otherObject также находится на стеке
useObject(obj)

...

staticField = heapification(otherObject)
```

- Оптимистичный статический анализ,

Оптимизации

Анализ эвакуаций

```
class SomeObject(var a: Int, var b: Double, c: String)

val obj = onStack(SomeObject(42, 42.0, "42"))
useField(obj.a)
useField(obj.b)
obj.c = "override"

otherObject.field = obj // otherObject также находится на стеке
useObject(obj)

...

staticField = heapification(otherObject)
```

- Оптимистичный статический анализ,
- Однако обладает издержками на дополнительную метаинформацию.

Оптимизации

Стратегии

1. Анализ утеканий

- Скаляризация
 - Локальный анализ,
 - У объекта отсутствуют использования по ссылке.
- Размещение на стеке
 - Локальный и межпроцедурный анализ,
 - У объекта отсутствуют утекания во всем графе вызовов;

2. Анализ эвакуаций

- Большое количество перемещаемых на стек объектов,
- Требуется эвакуации — операция глобального рекурсивного перемещения на кучу,
- Требуется дополнительная метainформация о расположении объектов на стеке.

Межпроцедурный анализ частичных утеканий

Идея

```
def lambdaUse(x: (Int => Unit)) = {  
  ...  
  if (globalFlag) { // Всегда = false  
    globalVar = x  
  }  
  ...  
}  
  
val lambda = { x => println(x) }  
lambdaUse(lambda)
```

Межпроцедурный анализ частичных утеканий

Идея

```
def lambdaUse(x: (Int => Unit)) = {  
  ...  
  if (globalFlag) { // Всегда = false  
    globalVar = evacuate(x)  
  }  
  ...  
}  
  
val lambda = onStack( { x => println(x) } )  
lambdaUse(lambda)
```

Межпроцедурный анализ частичных утеканий

Идея

```
def lambdaUse(x: (Int => Unit)) = {  
  ...  
  if (globalFlag) { // Всегда = false  
    globalVar = evacuate(x)  
  }  
  ...  
}  
  
val lambda = onStack( { x => println(x) } )  
lambdaUse(lambda)
```

- Будем перемещать объект на стек, если у него нет гарантированного *утекающего* использования,
- Если такие использования есть, то перед их исполнением будем вставлять операцию *эвакуация*, которая создаст копию объекта на куче.

Межпроцедурный анализ частичных утеканий

Оптимизации

1. Быстрые пути в эвакуации

```
def evacuation[T](x: T): T = {  
  ... // системный вызов  
}
```

Межпроцедурный анализ частичных утеканий

Оптимизации

1. Быстрые пути в эвакуации

```
def evacuation[T](x: T): T = {  
  if (x == null) {  
    return null  
  } else if (isOnHeap(x)) {  
    return x  
  } else {  
    ...    // системный вызов  
  }  
}
```

Межпроцедурный анализ частичных эвакуаций

Оптимизации

2. Дополнительный стековый слот для сохранения результата

```
val lambda = onStack( { x => println(x) } )
```

Межпроцедурный анализ частичных эвакуаций

Оптимизации

2. Дополнительный стековый слот для сохранения результата

```
val cachedLambda = null
val lambda = onStack( { x => println(x) } )
```

```
def evacuation[T](x: T): T = {
  ...
} else if (*(δx + 8) ≠ null) {
  return *(δx + 8)
} else {
  ... // системный вызов

  *(δx + 8) = result
  return result
}
```

Окружение

Многоязыковая виртуальная машина Huawei

1. Оптимизирующий статический компилятор,
2. Управляемая среда исполнения,
3. Поддержка языков Java, Scala, Cangjie.



Результаты

Статистика

Название теста	Анализ утеканий	Межпроцедурный анализ частичных утеканий
scala-std	463 + 103	397
Виртуальная машина Huawei	799 + 394	4488

Заключение

В ходе работы было сделано:

1. Изучены существующие подходы к уменьшению издержек на выделение объектов в куче,
2. Разработана и реализована оптимизация «межпроцедурный анализ частичных утеканий»,
3. Полученное решение было протестировано.

Направления дальнейшей работы

1. Расширить анализ на другие типы, помимо лямбда-функций,
2. Добавить поддержку использования профилировочной информации.

Спасибо за внимание

Анализ утеканий

Алгоритм

NoEscape



RetEscape

RcvEscape



RetRcvEscape



GlobalEscape

Межпроцедурный анализ частичных эвакуаций

Алгоритм

